SISTEMA DE BUSCA OTIMIZADO PARA CATÁLOGO DE PRODUTOS – MEGASTORE

---

1. Introdução

Com o crescimento exponencial do comércio eletrônico, empresas como a MegaStore enfrentam desafios significativos relacionados à organização e recuperação eficiente de informações. Com milhões de produtos disponíveis em seu catálogo, torna-se essencial implementar um sistema de busca rápido, preciso e escalável.

O presente trabalho tem como objetivo propor e implementar um sistema de busca otimizado utilizando estruturas de dados eficientes e algoritmos adequados, com foco na linguagem Rust. A solução visa melhorar a experiência do usuário, aumentar a taxa de conversão de vendas e garantir a escalabilidade do sistema.

---

2. Problema Identificado

O sistema atual da MegaStore apresenta as seguintes limitações:

- Lentidão nas buscas devido à ineficiência na indexação dos dados
- Resultados imprecisos, retornando produtos irrelevantes
- Baixa escalabilidade para lidar com grandes volumes de dados
- Experiência negativa do usuário, impactando diretamente as vendas

Esses problemas ocorrem principalmente pela ausência de estruturas de dados otimizadas e algoritmos eficientes para busca e organização das informações.

---

3. Análise dos Impactos

Os principais afetados pelo problema são:

- **Clientes:** dificuldade em encontrar produtos rapidamente
- **Empresa:** perda de vendas e redução da taxa de conversão
- **Plataforma:** aumento de carga computacional e baixa eficiência

A longo prazo, a não resolução desse problema pode comprometer a competitividade da MegaStore no mercado digital.

---

4. Solução Proposta

A solução desenvolvida baseia-se na combinação de duas abordagens principais:

4.1 Indexação com Tabela Hash (HashMap)

Utiliza-se a estrutura HashMap para indexar os produtos por diferentes critérios, como:

- Nome
- Categoria
- Marca

Essa estrutura permite acesso em tempo constante (O(1)), garantindo buscas extremamente rápidas.

4.2 Modelagem com Grafos

Os produtos são representados como vértices em um grafo, onde as conexões representam relações como:

- Produtos similares
- Produtos frequentemente comprados juntos
- Produtos da mesma categoria

Essa abordagem permite implementar recomendações inteligentes.

4.3 Algoritmo de Busca (BFS)

Para gerar recomendações, foi utilizado o algoritmo de Busca em Largura (BFS), que percorre o grafo de forma eficiente e retorna produtos relacionados ao item buscado.

---

5. Arquitetura do Sistema

O sistema foi estruturado de forma modular, seguindo boas práticas de engenharia de software:

- **product.rs:** definição da estrutura do produto
- **index.rs:** responsável pela indexação dos dados
- **graph.rs:** implementação do grafo de recomendações
- **search.rs:** lógica principal de busca e recomendação
- **main.rs:** execução do sistema

Fluxo de funcionamento:

1. O usuário realiza uma busca
2. O sistema consulta o HashMap
3. Os produtos são recuperados rapidamente
4. O grafo é utilizado para gerar recomendações
5. Os resultados são exibidos ao usuário

---

6. Estruturas de Dados Utilizadas

6.1 HashMap

- Permite acesso rápido aos dados
- Complexidade: O(1)
- Ideal para indexação

6.2 Grafos

- Representam relações entre produtos
- Permitem recomendações inteligentes

6.3 Vetores (Vec)

- Utilizados para armazenar listas de produtos
- Estrutura simples e eficiente

---

7. Algoritmos Utilizados

Busca em Largura (BFS)

- Utilizada para recomendação
- Percorre o grafo por níveis
- Garante retorno de produtos próximos

Indexação

- Realizada por meio de HashMap
- Otimiza o tempo de busca

---

8. Análise de Desempenho

Operação	Complexidade
Busca	O(1)
Inserção	O(1)
Recomendação	O(n)

Comparado ao sistema anterior, houve:

- Redução significativa no tempo de resposta
- Aumento na precisão dos resultados
- Melhor utilização de recursos computacionais

---

9. Escalabilidade

O sistema foi projetado para suportar:

- Crescimento contínuo do catálogo
- Alto volume de acessos simultâneos
- Integração com sistemas distribuídos

A utilização de estruturas eficientes garante que o desempenho se mantenha mesmo com grandes volumes de dados.

---

10. Integração com a Plataforma

A solução pode ser integrada facilmente ao sistema da MegaStore por meio de:

- APIs REST
- Integração com banco de dados
- Sistemas de recomendação existentes

Isso permite sua aplicação em ambientes reais de produção.

---

11. Métricas de Avaliação

Para medir a eficácia do sistema, podem ser utilizados os seguintes indicadores:

- Tempo médio de resposta
- Taxa de conversão

- Precisão dos resultados
- Taxa de cliques em recomendações

---

12. Considerações Finais

O sistema desenvolvido atende aos requisitos propostos, oferecendo uma solução eficiente, escalável e de alto desempenho para o problema da MegaStore.

A combinação de HashMap e Grafos mostrou-se extremamente eficaz, proporcionando:

- Buscas rápidas
- Resultados relevantes
- Recomendações inteligentes

Além disso, a utilização da linguagem Rust contribui para maior segurança, eficiência e controle de memória, tornando o sistema robusto e confiável.

---

13. Conclusão

Este projeto demonstra a importância da escolha adequada de estruturas de dados e algoritmos na construção de sistemas eficientes.

A solução proposta não apenas resolve os problemas atuais da MegaStore, mas também prepara a plataforma para desafios futuros, garantindo escalabilidade e competitividade no mercado digital.